TEMPORADA XIV

CI NICARAGUA · 2026

DOCUMENTO DE LECTURA
05

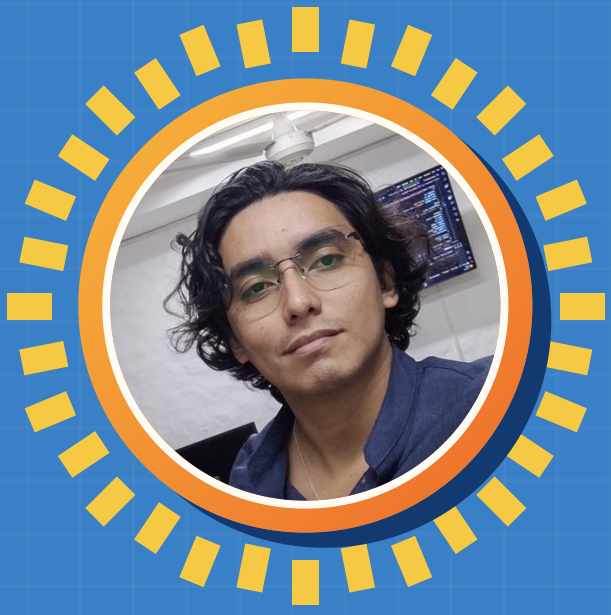
// Antes de la Unidad II · Control de versiones

TU CÓDIGO EN LA NUBE CON GIT Y GITHUB

DEL PRIMER COMMIT A TU LLAVE SSH

EN ESTA LECTURA

01 Git y GitHub no son lo mismo
02 Instalar y configurar Git
03 Tu primer repositorio local
04 Subir tu repositorio a GitHub
05 Conectarte con SSH
06 El trabajo de todos los días
07 Errores comunes
08 Para practicar



ELABORADO POR

MARVIN RAFAEL MEZA PINEDA

Facilitador · Centro Nacional de Innovación y Tecnología Francisco "El Chele" Moreno

Fundamentos de Desarrollo Backend con Java

Noviembre 2026

CAPÍTULO 01

1 GIT Y GITHUB NO SON LO MISMO

> **Git** guarda el historial de tu proyecto en tu computadora. **GitHub** es un sitio web donde guardas una copia de ese historial para respaldarlo, mostrarlo y compartirlo.

Seguro ya tuviste una carpeta con archivos como `proyecto_final`, `proyecto_final2` y `proyecto_final_AHORA_SI`. Git resuelve ese problema: guarda fotos de tu proyecto llamadas **commits**, y puedes volver a cualquiera de ellas. Además, todo equipo de desarrollo profesional trabaja con una herramienta así.

	Git	GitHub
Qué es	Un programa de control de versiones.	Un servicio web que aloja repositorios.
Dónde vive	En tu computadora.	En internet: github.com.
¿Necesita internet?	No.	Sí.
Lo usas con	Comandos en la terminal.	El navegador, y `git push` para subir.

Palabras que vas a escuchar

Palabra	Significado
Repositorio	Tu proyecto más todo su historial. También le dicen *repo*.
Commit	Una foto del proyecto en un momento, con un mensaje que explica qué cambió.
Rama	Una línea de trabajo. La principal se llama `main`.
Remoto	La copia del repositorio en otro lugar. Por costumbre se llama `origin`.

El recorrido de un cambio

Tus cambios pasan por cuatro lugares. Entender este dibujo es entender casi todo Git:



El **staging** es como la mesa donde acomodas lo que saldrá en la foto. Te permite elegir qué cambios entran en cada commit. Para traer a tu computadora lo que hay en GitHub, usarás `git pull`.

CAPÍTULO 02

2 INSTALAR Y CONFIGURAR GIT

Instalación en Windows

1. Entra a `git-scm.com`, descarga el instalador para Windows y ejecútalo. Puedes dejar las opciones por defecto. Incluye **Git Bash**, una terminal adicional.

2 También puedes instalarlo desde la terminal, igual que hiciste con el JDK:

```
PowerShell
> winget install Git.Git
```

3 Abre una terminal **nueva** y verifica. Tu número de versión puede ser otro:

```
Terminal
> git --version
git version 2.56.0.windows.4
```

Dile a Git quién eres

Cada commit lleva un nombre y un correo. Esta configuración se hace **una sola vez por computadora**:

```
Terminal
> git config --global user.name "Ana López"
> git config --global user.email "tu-correo@ejemplo.com"
> git config --global init.defaultBranch main
> git config --global --list          # para revisar lo que quedó
```

TU CORREO SERÁ PÚBLICO

En un repositorio público, cualquiera puede ver el correo de tus commits. Si prefieres no mostrarlo, en GitHub ve a **Settings** → **Emails** y activa **Keep my email addresses private**. Ahí mismo aparece una dirección `...@users.noreply.github.com`: usa esa en `user.email`.

Crea tu cuenta de GitHub

Entra a `github.com` y pulsa **Sign up**. Elige bien tu **nombre de usuario**: aparecerá en la dirección de tus proyectos y será parte de tu portafolio profesional. En esta lectura lo escribiremos como `TU-USUARIO`.

CAPÍTULO 03

3 TU PRIMER REPOSITORIO LOCAL

> Todo este capítulo ocurre en tu computadora, sin internet. Vamos a crear un proyecto llamado `practica-git`.

Paso 1: crear el repositorio

```
Terminal
> mkdir practica-git
> cd practica-git
> git init
Initialized empty Git repository in C:/Users/ana/practica-git/.git/
> code .                              # abre la carpeta en VS Code
```

`git init` crea una carpeta oculta llamada `.git`, donde vive todo el historial. **No la modifiques ni la borres**, y hazlo una sola vez por proyecto.

Paso 2: crear archivos y decidir qué ignorar

En VS Code, crea `Hola.java` con cualquier programa y compílalo con `javac Hola.java`. Crea también un archivo llamado exactamente `.gitignore`. Lo que escribas ahí, Git lo ignorará:

```
.gitignore
# Resultado de compilar: se genera de nuevo, no se sube
*.class

# Carpeta de compilación de Maven (la verás con Spring Boot)
target/
```

Paso 3: revisar, agregar y guardar

`git status` te dice cómo está todo. Úsalo todo el tiempo, porque nunca cambia nada:

```
Terminal
> git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        Hola.java
```

`Hola.class` no aparece: `.gitignore` ya está haciendo su trabajo. Ahora pasamos todo al staging y tomamos la foto:

```
Terminal
> git add .
> git commit -m "Primer programa en Java"
[main (root-commit) 2cbdf43] Primer programa en Java
 2 files changed, 2 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 Hola.java
> git log --oneline
2cbdf43 Primer programa en Java
```

Ese `2cbdf43` es el identificador del commit. El tuyo será distinto.

Comando	Qué hace
`git init`	Convierte la carpeta actual en un repositorio.
`git status`	Muestra qué cambió y qué está en staging.
`git add .`	Pasa al staging todos los cambios. `git add Hola.java` pasa solo ese archivo.
`git commit -m "mensaje"`	Guarda una foto de lo que está en staging.
`git log --oneline`	Muestra el historial, una línea por commit.
`git diff`	Muestra línea por línea lo que cambiaste y aún no agregas.

Mensajes de commit que sirven

✗ NO DICEN NADA

`cambios` `asdf` `ya funciona` `final final`

✓ EXPLICAN EL AVANCE

`Agrega clase Producto` `Valida precio negativo` `Corrige cálculo del IVA`

Haz commits pequeños y seguidos. Uno por cada avance que puedas describir en una frase.

CAPÍTULO 04

4 SUBIR TU REPOSITORIO A GITHUB

Paso 1: crear el repositorio vacío en GitHub

1. En la esquina superior derecha de GitHub, pulsa el botón **+** y luego **New repository**.
2. En **Repository name** escribe `practica-git`. Puedes agregar una descripción.
3. Elige la visibilidad **Public**.
4. **No agregues README, `.gitignore` ni licencia:** tu proyecto local ya tiene historial, y estos archivos crearían un conflicto en la primera subida.
5. Pulsa **Create repository**. GitHub te mostrará la dirección de tu repositorio.

Paso 2: conectar y subir

Terminal

```
> git remote add origin https://github.com/TU-USUARIO/practica-git.git
> git branch -M main
> git push -u origin main
To https://github.com/TU-USUARIO/practica-git.git
 * [new branch]      main → main
branch 'main' set up to track 'origin/main'.
```

Comando	Qué hace
`git remote add origin URL`	Le dice a tu repositorio dónde está su copia remota, y la llama `origin`.
`git branch -M main`	Se asegura de que tu rama se llame `main`.
`git push -u origin main`	Sube tus commits. `-u` recuerda el destino: en adelante basta con `git push`.
`git remote -v`	Muestra a qué dirección está conectado tu repositorio.

En la primera subida por HTTPS, Windows abrirá una ventana para que inicies sesión en GitHub desde el navegador. Al terminar, recarga la página de tu repositorio: ahí estará tu código.

PÚBLICO SIGNIFICA PÚBLICO

Nunca subas contraseñas, llaves ni datos personales. Borrar el archivo después no basta: **sigue en el historial**. Si te pasa con una contraseña, cámbiala de inmediato.

CAPÍTULO 05

5 CONECTARTE CON SSH

> Con **SSH**, tu computadora se identifica ante GitHub con un par de llaves criptográficas. Lo configuras una vez y luego subes código sin iniciar sesión en el navegador.

`id_ed25519`
LLAVE PRIVADA

Se queda en tu computadora. Es como la llave de tu casa: **no se comparte con nadie**, ni con el docente.

`id_ed25519.pub`
LLAVE PÚBLICA

Se copia a GitHub. Es como un candado: cualquiera puede verlo, pero solo tu llave lo abre.

	HTTPS	SSH
Dirección	`https://github.com/TU-USUARIO/repo.git`	`git@github.com:TU-USUARIO/repo.git`
Cómo te identificas	Inicio de sesión guardado o token.	Tu par de llaves.
Configuración	Ninguna al inicio.	Una vez por computadora.

Paso 1: ¿ya tienes llaves?

```
Terminal
> ls ~/.ssh
```

Si ves `id_ed25519` e `id_ed25519.pub`, ya tienes un par: salta al paso 3. Si la carpeta no existe o está vacía, sigue con el paso 2.

Paso 2: generar el par de llaves

Usa el correo de tu cuenta de GitHub. Responde a las preguntas así:

```
Terminal
> ssh-keygen -t ed25519 -C "tu-correo@ejemplo.com"
Generating public/private ed25519 key pair.
Enter file in which to save the key (C:\Users\ana/.ssh/id_ed25519):     # pulsa Enter
Enter passphrase (empty for no passphrase):                            # escribe una frase
Enter same passphrase again:                                           # repítela
Your identification has been saved in C:\Users\ana/.ssh/id_ed25519
Your public key has been saved in C:\Users\ana/.ssh/id_ed25519.pub
```

LA FRASE DE PASO

Protege tu llave si alguien entra a tu computadora. Mientras la escribes **no aparece nada en pantalla**: es normal. Si la olvidas no se puede recuperar, pero solo tendrías que generar otro par.

Paso 3: copiar la llave pública

Este comando copia la llave al portapapeles. Fíjate en que termina en `.pub`:

```
Terminal
> cat ~/.ssh/id_ed25519.pub | clip
```

Paso 4: agregarla a GitHub

1. En GitHub, pulsa tu foto de perfil y luego **Settings**.
2. En la sección **Access** de la barra lateral, pulsa **SSH and GPG keys** y luego **New SSH key**.
3. En **Title** escribe un nombre que identifique la computadora, como `Laptop de Ana`.
4. Deja **Key type** en **Authentication Key** y, en el campo **Key**, pega con `Ctrl` + `V`.
5. Pulsa **Add SSH key**. Si GitHub lo pide, confirma tu acceso.

Paso 5: probar la conexión

```
Terminal
> ssh -T git@github.com
The authenticity of host 'github.com (...)' can't be established.
ED25519 key fingerprint is SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU.
Are you sure you want to continue connecting (yes/no)? yes
Hi TU-USUARIO! You've successfully authenticated, but GitHub does not provide shell access.
```

Antes de escribir `yes`, compara la huella (*fingerprint*) de tu pantalla con la de este ejemplo: debe ser **idéntica**. Es la forma de confirmar que hablas con el GitHub real, y también está publicada en su documentación. Esa pregunta solo aparece la primera vez. El mensaje “does not provide shell access” es normal: si ves tu usuario, **funcionó**.

Paso 6: usar SSH en tus repositorios

Para un repositorio **que ya creaste con HTTPS**, como `practica-git`, cambia la dirección de `origin`:

```
Terminal
> git remote set-url origin git@github.com:TU-USUARIO/practica-git.git
> git remote -v
origin  git@github.com:TU-USUARIO/practica-git.git (fetch)
origin  git@github.com:TU-USUARIO/practica-git.git (push)
> git push
```

Para un repositorio **nuevo**, usa la dirección SSH desde el principio. En GitHub la encuentras en el botón **Code**, pestaña **SSH**:

```
Terminal
> git remote add origin git@github.com:TU-USUARIO/otro-proyecto.git
```

COMPUTADORAS COMPARTIDAS

En una computadora del laboratorio, tu llave queda al alcance de la siguiente persona. Al terminar, borra `id_ed25519` e `id_ed25519.pub` de la carpeta `.ssh` y quita la llave en **SSH and GPG keys** → **Delete**. Si en cualquier momento crees que alguien copió tu llave privada, haz lo mismo y genera un par nuevo.

Opcional: no escribir la frase en cada push

El **agente SSH** recuerda tu frase de paso. En una terminal de PowerShell **como administrador**, actívalo una vez. Luego agrega tu llave desde una terminal normal:

```
PowerShell
> Get-Service -Name ssh-agent | Set-Service -StartupType Manual     # como administrador
> Start-Service ssh-agent                                          # como administrador
> ssh-add ~/.ssh/id_ed25519                                        # terminal normal
```

Si usas Git Bash, los comandos son `eval "$(ssh-agent -s)"` y después `ssh-add ~/.ssh/id_ed25519`.

CAPÍTULO 06

6 EL TRABAJO DE TODOS LOS DÍAS

> Una vez configurado todo, trabajar con Git se reduce a repetir el mismo ciclo. Lo demás lo vas a necesitar de vez en cuando.

```
El ciclo de cada avance
> git status
> git add .
> git commit -m "Valida precio negativo"
> git push
```

Traer y descargar

Comando	Cuándo usarlo
`git pull`	Antes de empezar a trabajar: trae lo nuevo de GitHub a tu copia.
`git clone git@github.com:TU-USUARIO/repo.git`	Para descargar un repositorio completo, una sola vez. Crea una carpeta con su nombre.

Si trabajas en dos computadoras, el orden es: `git pull` al llegar y `git push` al irte. El repositorio de este curso lo clonas una vez y, antes de cada clase, solo haces `git pull`.

Deshacer lo más común

Comando	Qué hace
`git restore Hola.java`	Descarta los cambios sin agregar de ese archivo. **Se pierden para siempre.**
`git restore --staged Hola.java`	Lo saca del staging, pero conserva los cambios.
`git rm --cached "*.class"`	Deja de seguir los `.class` que subiste por error, sin borrarlos de tu carpeta. Después, haz commit y push.

Ramas: un primer vistazo

Una **rama** te deja probar algo sin tocar `main`. Si sale bien, la unes; si sale mal, `main` sigue intacta. En este curso verás ramas como `clase-03/poo-encapsulamiento-herencia`.

```
Terminal
> git switch -c mejora-menu                # crea la rama y entra en ella
> git add .
> git commit -m "Agrega opción de reporte"
> git push -u origin mejora-menu           # la primera vez, para esta rama
> git switch main                          # vuelve a main
> git merge mejora-menu                    # trae los cambios de la rama a main
> git push
```

7 CAPÍTULO 07 ERRORES COMUNES

> Git explica bastante bien lo que pasa, aunque sea en inglés. Lee el mensaje antes de copiar comandos de internet, y estos son los que más vas a encontrar:

Mensaje	Qué significa y qué hacer
`fatal: not a git repository`	No estás en la carpeta del proyecto. Entra con `cd`, o ejecuta `git init` si aún no es un repositorio.
`error: src refspec main does not match any`	Todavía no tienes ningún commit. Haz `git add .` y `git commit`.
`error: remote origin already exists.`	Ya hay un `origin`. Para cambiarlo, usa `git remote set-url origin URL`.
`git@github.com: Permission denied (publickey).`	GitHub no reconoce tu llave. Repite los pasos 3 y 4 del capítulo 5, y prueba con `ssh -T git@github.com`.
`! [rejected] main → main (fetch first)`	GitHub tiene commits que tú no tienes. Ejecuta `git pull` y luego `git push`. **No uses** `--force`.
`remote: Support for password authentication was removed`	GitHub ya no acepta tu contraseña en la terminal. Usa SSH.
`warning: LF will be replaced by CRLF`	No es un error: Windows y Git manejan distinto los saltos de línea. Puedes seguir.

Si la red bloquea SSH

Algunas redes de instituciones bloquean el puerto de SSH, y `ssh -T` se queda esperando hasta dar un error de tiempo agotado. GitHub ofrece una salida por el puerto 443. Crea el archivo `~/.ssh/config` con este contenido:

```
~/.ssh/config
Host github.com
    Hostname ssh.github.com
    Port 443
    User git
```

CAPÍTULO 08

8 PARA PRACTICAR

> Usa tu propia cuenta. Al terminar, deberías poder subir un proyecto a GitHub sin mirar esta lectura.

01 **Tu entorno** básico

Verifica `git --version`, configura tu nombre y tu correo, y revisa el resultado con `git config --global --list`.

02 **Tu primer repositorio** básico

Crea `practica-git` con un `.java` y un `.gitignore`. Haz al menos **3 commits** con mensajes claros y compruébalo con `git log --oneline`.

03 **A GitHub** básico

Crea el repositorio público en GitHub y sube tu proyecto. Ábrelo en una ventana de incógnito para confirmar que es público.

04 **Conexión SSH** intermedio

Genera tus llaves, agrega la pública a GitHub y logra el mensaje `Hi TU-USUARIO!`. Luego cambia `practica-git` a SSH y haz un `git push`.

05 **Dos copias** intermedio

Clona tu repositorio en otra carpeta, como si fuera otra computadora. Haz un commit y un push desde ahí, vuelve a la carpeta original y tráelo con `git pull`.

06 **Una rama completa** reto

Crea la rama `mejora-readme` y agrega un `README.md` con tu nombre y cómo ejecutar tu programa. Súbela, compruébala en GitHub, únela a `main` y sube el resultado.

Resumen del documento

Momento	Comandos	Recuerda
Una vez por PC	`git config --global` `ssh-keygen`	La llave privada no se comparte.
Una vez por proyecto	`git init` `git remote add` `git clone`	Crea primero el `.gitignore`.
Cada avance	`git status` `git add .` `git commit -m` `git push`	Commits pequeños, mensajes claros.
Antes de trabajar	`git pull`	Trae lo nuevo primero.
Para revisar	`git status` `git log --oneline` `git remote -v`	Nunca cambian nada.

Antes de la próxima clase

- Tener tu cuenta de GitHub y al menos un repositorio público con 3 commits.
- Haber logrado el mensaje `Hi TU-USUARIO!` con `ssh -T git@github.com`.
- Poder explicar la diferencia entre Git y GitHub, y entre tu llave privada y tu llave pública.
- Con esto ya puedes guardar tus proyectos de **Spring Boot**, el siguiente tema del curso.

Referencias

Chacon, S. y Straub, B. *Pro Git* (en español). git-scm.com/book/es/v2

GitHub Docs. *Crear un repositorio.* docs.github.com/repositories/creating-and-managing-repositories/creating-a-new-repository

GitHub Docs. *Generar una clave SSH.* docs.github.com/authentication/connecting-to-github-with-ssh/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agent

GitHub Docs. *Huellas de las claves SSH de GitHub.* docs.github.com/authentication/keeping-your-account-and-data-secure/githubs-ssh-key-fingerprints

GitHub Docs. *Usar SSH por el puerto HTTPS.* docs.github.com/authentication/troubleshooting-ssh/using-ssh-over-the-https-port



MARVIN RAFAEL MEZA PINEDA

Facilitador · Fundamentos de Desarrollo Backend con Java

Centro Nacional de Innovación y Tecnología Francisco “El Chele” Moreno · Temporada XIV · 2026